CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 2 Divide-and-Conquer

Midterm summary · Insertion Sort · Merge Sort

Muharib

What's inside

01 The idea

02 Insertion sort

03 Loop invariant

04 Merge sort

05 Analysis of merge sort

06 n log n vs n^2

07 Merge sort vs quick sort

08 Review sheet

01 The idea

Divide and conquer is a way to design fast algorithms. We break a big problem into smaller problems of the same form, solve the small ones, and then combine their solutions.

#	Step	What it does
1	**Divide**	Break the problem into sub-problems of the same form as the original.
2	**Conquer**	Solve the sub-problems independently, usually recursively.
3	**Combine**	Combine the sub-problem solutions into the answer to the original problem.

The two classic examples are merge sort and quicksort.

What the midterm takes from it

A true/false statement about the running-time equation of **MERGESORT** appeared in two midterms. There is also an MCQ that separates divide and conquer from dynamic programming and greedy.

02 Insertion sort

We insert the items one at a time. We start from the second element and pull it back until it

is in the right place.

```
InsertionSort(A)
    for i ← 2 to length(A) do
        key ← A[i]
        j ← i − 1
        while j > 0 and A[j] > key do
            A[j + 1] ← A[j]
            j ← j − 1
        A[j + 1] ← key
```

The lecture example on `6 4 3 8 5`:

i	key	Before	After
2	4	6 4 3 8 5	4 6 3 8 5
3	3	4 6 3 8 5	3 4 6 8 5
4	8	3 4 6 8 5	3 4 6 8 5
5	5	3 4 6 8 5	3 4 5 6 8

Case	Input	Time
Best case	Already sorted. The while loop never iterates.	$O(n)$
Worst case	Reverse sorted. The while loop goes all the way back each time.	$O(n^2)$

Why n^2? The outer loop runs $n - 1$ times, and in the worst case the inner loop runs about n times.

03 Loop invariant

A loop invariant is something that should be true at every iteration. We use it to prove that an algorithm is correct. For insertion sort: "before iteration i, the first $i - 1$ elements are

sorted".

Step	Meaning
Initialization	The invariant holds before the first iteration.
Maintenance	If it is true before iteration `t`, it is true before iteration `t + 1`.
Termination	It is true at the end of the last iteration, which gives the result: the whole array is sorted.

InsertionSort is an algorithm that correctly sorts an arbitrary n-element array in time about n^2.

04 Merge sort

```
MERGESORT(A):
    n ← length(A)
    if n ≤ 1:
        return A                          already sorted
    L ← MERGESORT(A[1 : n/2])             sort the left half
    R ← MERGESORT(A[n/2 + 1 : n])         sort the right half
    return MERGE(L, R)                    merge the two halves
```

The lecture example on `6 4 3 8 1 5 2 7`:

Divide	[6 4 3 8 1 5 2 7]
	[6 4 3 8] [1 5 2 7]
	[6 4] [3 8] [1 5] [2 7]
	[6] [4] [3] [8] [1] [5] [2] [7]
Merge	[4 6] [3 8] [1 5] [2 7]
	[3 4 6 8] [1 2 5 7]
	[1 2 3 4 5 6 7 8]

MERGE takes two sorted arrays and walks over them with two pointers, taking the smaller head each time. Its time is linear in the total length.

Why does it work? By the recursion invariant: "in every recursive call, **MERGESORT** returns a sorted array".

Step	In merge sort
Base case	`n = 1`: a one-element array is always sorted.
Maintenance	If `L` and `R` are sorted, then `MERGE(L, R)` is sorted.
Termination	The top recursive call returns the whole array sorted.

Merge sort runs in O(n log n) time in all the cases.

05 Analysis of merge sort

```
T(n) = T(n/2) + T(n/2) + T(MERGE)
```

```
T(n) = time to run MERGESORT on a list of size n
```

That is: the time to sort the left half, plus the time to sort the right half, plus the time to merge.

The midterm statement

> The time to run MERGESORT on a list of size n, T(n) = T(n/2) + T(n/2) + T(MERGE(n/2))

The midterm changed the last term: 472 wrote `T(MERGE(n/2))` and 451 wrote `T(MERGE/2)`. The slide writes only `T(MERGE)`, and the merge works on `n` elements (two halves of `n/2` each). So the statement is most likely **false** in both forms. I have no official answer key, so confirm with the instructor.

The recursion tree: we draw every recursive call as a node and count the work at each level.

Level	# problems	Size of each	Work at this level
0	1	n	6n
1	2	n/2	6n
2	4	n/4	6n
t	2^t	$n/2^t$	6n
log(n)	n	1	6n

```
Work per level   =  2^t problems × 6(n/2^t)  =  6n
```

```
Number of levels =  log(n) + 1
```

```
Total            =  6n(log(n) + 1)   →   O(n log n)
```

The `6n` is the merging work for the whole level. The 6 is a constant from counting the operations of **MERGE**. What matters is that it is linear.

06 n log n vs n²

`log(n)` asks: how many times do we divide `n` by 2 to get down to 1? All logarithms in

this course are base 2.

```
32 → 16 → 8 → 4 → 2 → 1          log(32) = 5
```

```
log(64) = 6     log(128) = 7     log(256) = 8     log(512) = 9
```

n	n log(n)	n²
8	24	64
16	64	256
32	160	1024
64	384	4096
128	896	16384
256	2048	65536
512	4608	262144
1024	10240	1048576

`log(n)` grows very slowly, which is why `n log n` is far better than `n²` for large `n`.

07 Merge sort vs quick sort

Both are divide and conquer, both are recursive, and both sort by comparison. The

differences:

	Merge sort	Quick sort
Splitting	Always in the middle, into two equal halves.	Around a pivot, and the split may not be equal.
Auxiliary array	Required.	Not required.
Merge step	Required.	Not required.
Best / Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n \log n)$	$O(n^2)$

The idea of quick sort: pick a pivot, put the smaller elements on its left and the larger on its right (partition), then sort each side the same way.

08 Review sheet

Question	Answer
Steps of divide and conquer?	Divide → Conquer → Combine
Insertion sort	Best $O(n)$ · Worst $O(n^2)$
Merge sort	$O(n \log n)$ in all cases
Merge sort equation?	$T(n) = T(n/2) + T(n/2) + T(\text{MERGE})$
Levels of the tree?	$\log(n) + 1$
Level `t`?	2^t problems of size $n/2^t$
Steps of a loop invariant?	Initialization · Maintenance · Termination
Worst case of quick sort?	$O(n^2)$